Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М80-211БВ-25

Студент: Князев Александр Владимирович

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 15.10.26

Москва, 2026

Постановка задачи

Вариант 6

Разработать и отладить две программы на языке Си для Linux, реализующие взаимодействие родительского и дочернего процессов через неименованный канал pipe. Для создания дочернего процесса использовать fork, для запуска отдельной программы child - execve.

Первой строкой пользователь вводит имя входного файла. Родитель открывает его на чтение и перенаправляет стандартный ввод дочернего процесса на этот файл. Каждая строка содержит произвольное количество целых чисел типа int. Дочерний процесс вычисляет сумму чисел отдельно для каждой строки и передаёт результаты родителю через pipe. Родитель выводит полученные суммы в стандартный поток вывода.

Ввод и вывод данных выполняются через read и write. Необходимо проверять результаты системных вызовов, закрывать ненужные файловые дескрипторы и получать статус завершения дочернего процесса через waitpid.

Общий метод и алгоритм решения

Организация процессов и данных

Программа состоит из parent.c и child.c. Схема движения данных: входной файл → стандартный ввод child → вычисление суммы строки → стандартный вывод child → pipe → parent → терминал. По каналу передаётся текст: десятичная запись каждой суммы завершается символом перевода строки.

Каждое входное число проверяется на принадлежность диапазону int. Переменные number и sum имеют тип long long; перед добавлением числа проверяется переполнение суммы. Поэтому результат строки может превышать INT_MAX, например 2147483647 + 1 = 2147483648.

Создание процесса и перенаправление потоков

Родитель читает имя файла через read до перевода строки или конца ввода. Буфер вмещает до 4095 байт имени и завершающий нулевой символ; пустое имя и нулевой байт внутри имени отклоняются. Затем open с флагом O_RDONLY открывает файл, а pipe создаёт дескрипторы чтения pipefd[0] и записи pipefd[1].

После fork дочерний процесс закрывает конец чтения канала. Вызов dup2(filefd, STDIN_FILENO) связывает его стандартный ввод с файлом, а dup2(pipefd[1], STDOUT_FILENO) направляет стандартный вывод в канал. Исходные дескрипторы закрываются, если они отличаются от целевых. execve("./child", args, environ) загружает программу child с сохранёнными перенаправлениями. При ошибке execve ребёнок завершается через _exit(127).

Чтение и разбор входных данных

Ребёнок читает данные блоками по 4096 байт. Переменные negative, in_number, has_digit и has_values сохраняют состояние между вызовами read, поэтому число и строка могут пересекать границу буфера. Поддерживаются знаки + и -, пробелы, табуляция и окончания строк CRLF. Пустые строки и строки только из разделителей не создают результата.

При завершении числа его значение добавляется к сумме. При переводе строки print_number преобразует сумму в текст и передаёт её через write. На конце файла разбор выполняется так, как если бы поступил перевод строки: последняя непустая строка обрабатывается и без завершающего символа \n. Время обработки линейно по объёму входа; размер рабочего буфера постоянен.

Передача результатов и завершение

Родитель закрывает входной файл и собственный конец записи pipe, затем читает канал и копирует полученные байты в stdout. Он не разбирает суммы повторно: один read может получить часть результата либо несколько строк сразу. Функция write_all учитывает частичную запись; прерванные read, write и waitpid повторяются при errno == EINTR.

После завершения ребёнка и закрытия всех концов записи read родителя возвращает 0. Поэтому закрытие лишнего конца записи обязательно для получения конца потока. Родитель закрывает конец чтения, вызывает waitpid и проверяет WIFEXITED и WEXITSTATUS. Ожидание выполняется после чтения канала, чтобы ребёнок не остановился на записи в заполненный pipe, пока родитель ждёт его завершения.

При ошибке разбора ребёнок возвращает 1 без отдельного сообщения; родитель сообщает Child failed и также возвращает 1. Уже переданные суммы предыдущих строк при этом могут оставаться в stdout. Подсказка Enter filename и сообщения родителя выводятся в stderr, отдельно от результатов.

Код программы

Ниже приведены полные исходные тексты parent.c и child.c. Код соответствует представленной реализации с обменом через pipe.

parent.c

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stddef.h>
#include <errno.h>

extern char **environ;

static int write_all(int fd, const char *buffer, size_t length)
{
    size_t sent = 0;
    while (sent < length) {
        ssize_t n = write(fd, buffer + sent, length - sent);
        if (n == -1 && errno == EINTR) {
            continue;
        }
        if (n <= 0) {
            return -1;
        }
        sent += (size_t)n;
    }
    return 0;
}

static void report_error(const char *message)
{
    size_t length = 0;
    while (message[length] != '\0') {
        length++;
    }
    write_all(STDERR_FILENO, message, length);
}

int main(void){
    const char prompt[] = "Enter filename: ";
    if (write_all(STDERR_FILENO, prompt, sizeof(prompt) - 1) == -1) {
        return 1;
    }

    char filename[4096];
    size_t length = 0;

    while (1) {
        char c;
        ssize_t n = read(STDIN_FILENO, &c, 1);
        if (n == -1 && errno == EINTR) {
            continue;
        }
        if (n == -1) {
            report_error("Failed to read filename\n");
            return 1;
        }
        if (n == 0 || c == '\n') {
            break;
        }
        if (length == sizeof(filename) - 1) {
            report_error("Filename is too long\n");
            return 1;
        }
```

```
    if (c == '\0') {
        report_error("Invalid filename\n");
        return 1;
    }
    filename[length++] = c;
}
if (length == 0) {
    report_error("Filename is empty\n");
    return 1;
}
filename[length] = '\0';

int filefd = open(filename, O_RDONLY);
if (filefd == -1) {
    report_error("Cannot open file\n");
    return 1;
}

int pipefd[2];
if (pipe(pipefd) == -1) {
    report_error("Cannot create pipe\n");
    close(filefd);
    return 1;
}

pid_t pid = fork();
if (pid == -1) {
    report_error("Cannot fork\n");
    close(filefd);
    close(pipefd[0]);
    close(pipefd[1]);
    return 1;
}

if (pid == 0) {
    if (close(pipefd[0]) == -1) {
        report_error("Cannot close child pipe read end\n");
        _exit(1);
    }

    if (dup2(filefd, STDIN_FILENO) == -1) {
        report_error("Cannot redirect stdin\n");
        _exit(1);
    }
    if (filefd != STDIN_FILENO && close(filefd) == -1) {
        report_error("Cannot close child input file\n");
        _exit(1);
    }

    if (dup2(pipefd[1], STDOUT_FILENO) == -1) {
        report_error("Cannot redirect stdout\n");
        _exit(1);
    }
    if (pipefd[1] != STDOUT_FILENO && close(pipefd[1]) == -1) {
        report_error("Cannot close child pipe write end\n");
        _exit(1);
    }

    char *const args[] = {"./child", NULL};
    execve("./child", args, environ);
    report_error("Cannot execute child\n");
    _exit(127);
}

int failed = 0;
if (close(filefd) == -1) {
    report_error("Cannot close input file\n");
```

```
            failed = 1;
        }
        if (close(pipefd[1]) == -1) {
            report_error("Cannot close pipe write end\n");
            failed = 1;
        }

        char buffer[4096];
        if (!failed) {
            while (1) {
                ssize_t n = read(pipefd[0], buffer, sizeof(buffer));
                if (n == -1 && errno == EINTR) {
                    continue;
                }
                if (n == -1) {
                    report_error("Cannot read pipe\n");
                    failed = 1;
                    break;
                }
                if (n == 0) {
                    break;
                }
                if (write_all(STDOUT_FILENO, buffer, (size_t)n) == -1) {
                    report_error("Cannot write output\n");
                    failed = 1;
                    break;
                }
            }
        }

        if (close(pipefd[0]) == -1) {
            report_error("Cannot close pipe read end\n");
            failed = 1;
        }

        int status;
        pid_t result;
        do {
            result = waitpid(pid, &status, 0);
        } while (result == -1 && errno == EINTR);

        if (result == -1) {
            report_error("Cannot wait for child\n");
            return 1;
        }
        if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
            report_error("Child failed\n");
            return 1;
        }
        if (failed) {
            return 1;
        }
        return 0;
    }
```

child.c

```
    #include <unistd.h>
    #include <errno.h>
    #include <limits.h>

    static int write_all(int fd, const char *buffer, size_t length){
        size_t sent = 0;
        while (sent < length) {
            ssize_t n = write(fd, buffer + sent, length - sent);

            if (n == -1 && errno == EINTR) {
                continue;
```

```
        }
        if (n <= 0) {
            return -1;
        }
        sent += (size_t)n;
    }
    return 0;
}

static int print_number(long long value){
    char reversed[32];
    int length = 0;
    long long current = value;

    do {
        int digit = (int)(current % 10);
        if (digit < 0) {
            digit = -digit;
        }
        reversed[length] = (char)('0' + digit);
        length++;
        current /= 10;
    } while (current != 0);

    if (value < 0) {
        reversed[length] = '-';
        length++;
    }

    char output[32];

    for (int i = 0; i < length; i++) {
        output[i] = reversed[length - 1 - i];
    }

    output[length] = '\n';

    if (write_all(STDOUT_FILENO, output, (size_t)length + 1) == -1) {
        return 1;
    }
    return 0;
}

int main(void){

    char buffer[4096];
    long long sum = 0;
    long long number = 0;

    int negative = 0;
    int in_number = 0;
    int has_digit = 0;
    int has_values = 0;

    while (1) {
        ssize_t n = read(STDIN_FILENO, buffer, sizeof(buffer));

        if (n == -1 && errno == EINTR) {
            continue;
        }
        if (n == -1) {
            return 1;
        }
        int end_of_file = 0;
        ssize_t count = n;

        if (n == 0) {
```

```
            end_of_file = 1;
            count = 1;
        }
        for (ssize_t i = 0; i < count; i++) {
            char c;

            if (end_of_file) {
                c = '\n';
            } else {
                c = buffer[i];
            }
            if (c >= '0' && c <= '9') {
                if (!in_number) {
                    in_number = 1;
                    number = 0;
                    negative = 0;
                    has_digit = 0;
                }

                int digit = c - '0';
                long long limit = INT_MAX;

                if (negative) {
                    limit = -(long long)INT_MIN;
                }
                if (number > (limit - digit) / 10) {
                    return 1;
                }

                number = number * 10 + digit;
                has_digit = 1;
            }
            else if (c == '-' || c == '+') {
                if (in_number) {
                    return 1;
                }

                in_number = 1;
                number = 0;
                has_digit = 0;
                negative = 0;

                if (c == '-') {
                    negative = 1;
                }
            }
            else if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
                if (in_number) {
                    if (!has_digit) {
                        return 1;
                    }
                    long long value = number;

                    if (negative) {
                        value = -number;
                    }
                    if (value > 0 && sum > LLONG_MAX - value) {
                        return 1;
                    }
                    if (value < 0 && sum < LLONG_MIN - value) {
                        return 1;
                    }

                    sum += value;
                    has_values = 1;
                    in_number = 0;
                    has_digit = 0;
```

```
                }
                if (c == '\n' && has_values) {
                    if (print_number(sum) != 0) {
                        return 1;
                    }
                    sum = 0;
                    has_values = 0;
                }
            }
            else {
                return 1;
            }
        }
        if (end_of_file) {
            break;
        }
    }
    return 0;
}
```

Протокол работы программы

Сборка и запуск

Ниже приведены воспроизводимые контрольные примеры и ожидаемые результаты. Они не являются протоколом личных запусков студента. Команды выполняются в Linux из каталога, содержащего parent.c и child.c.

```
gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    parent.c -o parent
gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    child.c -o child
```

Обе исполняемые программы должны находиться в текущем каталоге: parent запускает ребёнка по относительному пути ./child. Имя входного файла вводится отдельной строкой. Для автоматического запуска его можно передать через printf.

Основной пример

Создание numbers.txt и запуск:

```
printf '10 20 30\n5 15 25\n100 200 300\n-10 5 3\n2147483647 1\n' \
    > numbers.txt
printf 'numbers.txt\n' | ./parent
```

Ожидаемый стандартный вывод:

```
60
45
600
-2
2147483648
```

Ожидаемый код завершения - 0. Подсказка Enter filename: относится к stderr и не входит в показанный стандартный вывод.

Знаки и границы int

```
printf '+7 -7 0\n-2147483648 2147483647\n' > signs.txt
printf 'signs.txt\n' | ./parent
```

Ожидаемый stdout: две строки 0 и -1; код завершения 0. Пример проверяет явный знак плюса, отрицательные числа и границы 32-битного int в Linux.

Пустые строки и окончания CRLF

```
printf '\n \t\r\n1\t2\r\n\n3 4\n' > spaces.txt
printf 'spaces.txt\n' | ./parent
```

Ожидаемый stdout: две строки 3 и 7; код завершения 0. Строки без чисел пропускаются.

Последняя строка без перевода строки

```
printf '8 -3 2' > last.txt
printf 'last.txt\n' | ./parent
```

Ожидаемый stdout: строка 7 с завершающим переводом строки; код завершения 0.

Пустой файл

```
: > empty.txt
printf 'empty.txt\n' | ./parent
```

Ожидаемый stdout пуст; код завершения 0.

Строка длиннее рабочего буфера

```
awk 'BEGIN { for (i=0; i<10000; i++) printf "1 ";
             printf "\n"; }' > long.txt
printf 'long.txt\n' | ./parent
```

Входная строка содержит 10 000 единиц и превышает 4096 байт. Ожидаемый stdout: строка 10000; код завершения 0.

Некорректные данные

```
printf '1 x 2\n' > bad.txt
printf 'bad.txt\n' | ./parent
printf 'exit=%s\n' "$?"
```

Ожидается пустой stdout программы, сообщение Child failed в stderr после подсказки и код завершения 1. Последняя команда оболочки отдельно печатает exit=1. Такое же завершение ожидается для файла со строкой 2147483648 либо строкой, содержащей только знак минуса.

Ошибка открытия файла

Если ввести имя несуществующего файла, ожидаются сообщение Cannot open file в stderr и код завершения 1. Дочерний процесс при этом не создаётся.

Трассировка взаимодействия процессов

Для самостоятельного наблюдения системных вызовов можно выполнить следующую команду при установленном strace:

```
printf 'numbers.txt\n' | strace -f -s 160 \
    -e trace=process,desc -o trace.txt ./parent
```

Ключ -f включает дочерний процесс. При анализе trace.txt следует найти открытие входного файла, создание pipe, создание дочернего процесса, оба перенаправления dup2, запуск ./child через execve, чтение файла, запись результатов в канал, чтение родителем и ожидание завершения ребёнка. Конкретные PID, номера дополнительных дескрипторов и разбиение данных на вызовы read/write зависят от запуска.

В этом разделе приведена команда для получения трассировки. Фрагменты вывода strace, значения PID и результаты системных вызовов не подставляются: их следует брать из реально полученного файла trace.txt.

Вывод

В лабораторной работе реализовано взаимодействие родительского и дочернего процессов через неименованный канал pipe. Дочерний процесс вычисляет сумму целых чисел в каждой непустой строке входного файла, а родитель принимает и выводит результаты. Для управления процессами и потоками данных использованы fork, execve, open, dup2, pipe, waitpid, read и write.

Основные трудности были связаны с назначением файловых дескрипторов, перенаправлением потоков через dup2 и межпроцессным взаимодействием. В процессе работы удалось разобраться, какие концы канала необходимо закрывать и почему сначала нужно прочитать результаты, а затем ожидать завершения ребёнка. Разобраны обработка частичной записи, конца файла и ошибок входных данных.